Logged in

Select Mode

Preventive Mode

Logout

📊 DQDash - AI Data Quality Assistant

🛡 Preventive Mode - AI Database Design Advisor

Describe your business idea and the system will suggest an optimized database schema.

Enter Business Description

The system should support:
- subscribers
- sports channels
- channel owners
- live sports events
- subscription plans
- payments
- viewing history

Select LLM Model

phi3

Generate Database Design

📋 AI Generated Database Design

⚠ This AI-generated schema is a draft recommendation. Final validation by a database architect is recommended.

Clear Result

Database Design

Database Design**

Tables and Columns with Data Types:

- Subscribers (SubID, Name VARCHAR(255), Email VARCHAR(255), PlanID INT, FOREIGN KEY(PlanID) REFERENCES PaymentPlans(PaymentPlanID))
- SportsChannels (ChannelID, OwnerEmail VARCHAR(255), CategoryEnum ENUM('Basketball', 'Soccer', 'Football', 'Tennis'), SubscriptionStatus BOOLEAN DEFAULT FALSE)
- ChannelOwners (OwnerID INT AUTO_INCREMENT PRIMARY KEY, Email VARCHAR(255), Name VARCHAR(255))
- LiveEvents (EventID INT AUTO0_INCREMENT PRIMARY KEY, Date DATETIME, StartTime TIMESTAMP, EndTime TIMESTAMP, ChannelID INT, FOREIGN KEY(ChannelID) REFERENCES SportsChannels(ChannelID), EventStatus ENUM('InProgress', 'Upcoming'))
- SubscriptionPlans (PaymentPlanID INT AUTO_INCREMENT PRIMARY KEY, Name VARCHAR(255), Price DECIMAL(10,2))
- Payments (PaymentID INT AUTO_INCREMENT PRIMARY KEY, SubscriberEmail VARCHAR(255) REFERENCES Subscribers(Email), PlanID INT FOREIGN KEY(PlanID) REFERENCES SubscriptionPlans(PaymentPlanID), Amount DECIMAL(10,2), DatePaid DATE, Status ENUM('Pending', 'Completed'))
- ViewingHistory (ViewingID INT AUTO_INCREMENT PRIMARY KEY, EventID INT FOREIGN KEY(EventID) REFERENCES LiveEvents(EventID), SubscriberEmail VARCHAR(255) FOREIGN KEY(SubscriberEmail) REFERENCES Subscribers(Email), DateTime DATETIME, Status ENUM('Watched', 'Skipped'))

- JunctionTableForHistory (ViewingHistoryID INT AUTO_INCREMENT PRIMARY KEY, EventID INT FOREIGN KEY(EventID) REFERENCES LiveEvents(EventID), SubscriberEmail VARCHAR(255) FOREIGN KEY(SubscriberEmail) REFERENCES Subscribers(Email))

Normalization Explanation:

- Each table has a primary key ensuring uniqueness for each record.
- Foreign keys create relationships between tables, establishing the connections necessary to maintain referential integrity within the database without introducing redundancy (i.e., avoids data duplication). The use of junction tables helps manage many-to-many relations such as Subscribers and ViewingHistory efficiently by creating an additional table that holds foreign keys from both sides, allowing multiple relationships per subscriber to events.

**

SQL Script

```
, Category ENUM('Basketball', 'Soccer', 'Football', 'Tennis'), SubscriptionStatus BOOLEAN DEFAULT FALSE);
 INT REFERENCES PaymentPlans(PaymentPlanID));
elID INT REFERENCES SportsChannels(ChannelID), EventStatus ENUM('InProgress', 'Upcoming'));

'lanID INT REFERENCES SubscriptionPlans(PlanID), Amount DECIMAL(10,2) NOT NULL, DatePaid DATE DEFAULT CURRENT_DATE(), Status ENUM('Pending', 'Completed'));
ERENCES Subscribers(Email), DateTime DATETIME NOT NULL, Status ENUM('Watched', 'Skipped'));
ICES LiveEvents(EventID), SubscriberEmail VARCHAR(255) FOREIGN KEY(SubscriberEmail) REFERENSCEsSUBSCRIBER (Email));

nction is a junction table that allows for many subscribers per event through the combination of viewing events and user details without duplication or redundancy, mainta
```

ER Diagram (Text)

ER Diagram Description** Subscriber --(1
)--> Payments ChannelOwner --(M
)--> SportsChannels SportsEvents - (0,N)-->(LiveEvents) | LiveEvents -(1:)---> ViewingHistoryJunction ViewingHistoryJunction -(1:) --> Subscribers

⬇ Download SQL Schema

⬇ Download ER Diagram Text

⬇ Download Design Report

⬇ Download Options

⬇ Download Full Architecture Report